
IE3034 - Control System Engineering

3rd year 1st semester – 2026

Group Project

# Design and Performance Analysis of a PID-Controlled DC Motor Stabilization System using Raspberry Pi

**Members: Group 04**

- K K H Hansana - IT23690684
- K A K Fernando- IT23632646
- H D P Kurukuladithya - IT23689862
- R P Hettiarachchi - IT23690134

# Contents

# 1. Introduction

DC motors are widely used in industrial and embedded systems due to their simple structure, reliability, and ease of control. Applications such as conveyor systems, robotics, and automated manufacturing require precise control of motor speed to ensure efficiency and accuracy.

However, controlling the speed of a DC motor is not straightforward because of system nonlinearities, load variations, and disturbances. Without proper control, the motor may exhibit undesirable behaviors such as overshoot, oscillations, or slow response.

To overcome these issues, feedback control systems are used. One of the most widely adopted techniques is the PID (Proportional–Integral–Derivative) controller, which continuously adjusts the motor input based on the error between the desired and actual speed. This allows the system to achieve fast response, minimal steady-state error, and stable operation.

In modern embedded systems, microcontrollers and single-board computers such as the Raspberry Pi are commonly used to implement such control algorithms in real time.

## 1.2 Objectives of the Assignment

To design, implement, and evaluate a PID (Proportional-Integral-Derivative) controller for a DC motor drive system using a Raspberry Pi, with the primary goal of achieving motor speed stabilization in the minimum possible time. Students will apply control systems theory, hardware interfacing, PCB design, and experimental analysis through an integrated practical assignment.

- To develop a hardware system capable of controlling a DC motor using a motor driver and encoder feedback
- To design and fabricate a custom PCB integrating all required components
- To implement a real-time PID control algorithm for motor speed regulation
- To minimize the motor settling time while satisfying performance constraints such as:
  - Maximum overshoot $\leq 15\%$
  - Steady-state error $\leq 2\%$
  - No sustained oscillations
- To experimentally evaluate system performance using step response analysis
- To analyze the effect of different PID gain values and identify the optimal controller configuration

### 1.3 Overview of the System

The developed system consists of a Raspberry Pi-based closed-loop motor control system designed to regulate the speed of a DC motor.

The key components of the system include:

- **Raspberry Pi 4** – acts as the main controller and executes the PID algorithm
- **TB6612FNG motor driver module**– controls motor direction and speed using PWM signals
- **6V N20 DC motor with encoder** – provides mechanical output and speed feedback
- **Custom-designed PCB** – integrates power distribution, signal routing, and supporting components such as resistors and capacitors

## 2. Hardware Design

### 2.1 Component Selection

The hardware components for this system were selected based on performance requirements, compatibility, availability, and ease of integration with the Raspberry Pi.

#### 2.1.1 Raspberry Pi 4

The Raspberry Pi 4 acts as the central control unit of the system. It executes the PID control algorithm and generates PWM signals to control the motor speed.

It also reads encoder signals through GPIO pins to calculate the motor speed in real time. The GPIO pins operate at 3.3 V logic levels, making it essential to ensure proper interfacing with external components.

The Raspberry Pi was selected due to:

- High computational capability
- Availability of multiple GPIO pins
- Support for Python-based control implementation
- Built-in libraries for GPIO and PWM control

### 2.1.2 DC Motor with Encoder

A 6 V N20 brushed DC motor with an integrated quadrature encoder (300 RPM) was used as the actuator in this system. This type of motor is compact, lightweight, and commonly used in robotics and precision control applications.

The motor converts electrical energy into rotational motion, while the attached encoder provides real-time feedback of the motor shaft movement. The encoder generates two pulse signals (C1 and C2) with a phase difference, allowing both speed and direction of rotation to be determined.

The selection of this motor was based on the following factors:

- Low voltage operation : Suitable for safe laboratory testing and compatible with available power supplies
- Moderate speed (300 RPM): Provides a controllable speed range for PID tuning and performance analysis
- Integrated encoder: Eliminates the need for external sensing devices and simplifies system design
- Compact size (N20 form factor): Easy integration into the experimental setup

### 2.1.3 TB6612FNG Motor Driver

The TB6612FNG is a MOSFET-based dual H-bridge motor driver used to control the DC motor in this system. It was selected due to its high efficiency, low voltage drop, and compact size, making it more suitable than traditional drivers.

The driver uses separate power supplies:

- VCC (3.3 V) from the Raspberry Pi for logic
- VM from the external motor supply

These share a common ground, but their positive lines must remain isolated to protect the Raspberry Pi.

Motor control is achieved using:

- PWMA (PWM signal) for speed control
- AIN1 and AIN2 for direction control

The STBY pin is connected to 3.3 V, keeping the driver always enabled and preventing unstable behavior.

Overall, the TB6612FNG provides an efficient and reliable interface for PWM-based motor control in the PID system.

### 2.1.4 Power Supply System

The system uses a dual power supply configuration to ensure safe and stable operation of both control and motor circuits. The Raspberry Pi and logic components are powered by a regulated 5 V supply, while the motor is powered separately through the TB6612FNG motor driver using an external supply (VM).

Within the driver, the logic voltage (VCC) is connected to the Raspberry Pi's 3.3 V rail, ensuring compatibility with GPIO signal levels. The motor voltage (VM) is supplied independently to provide sufficient power for motor operation.

Both power supplies share a common ground, which is essential for proper signal reference. However, the positive lines of the logic and motor supplies are kept isolated to prevent damage to sensitive components.

This separation of power domains reduces electrical noise and improves overall system reliability during operation.

## 3. PCB Design & Implementation

### 3.1 Schematic Design

The circuit schematic was designed using an EDA tool (EasyEDA) and organized into functional sections including power supply, motor driver, encoder interface, and Raspberry Pi connections.

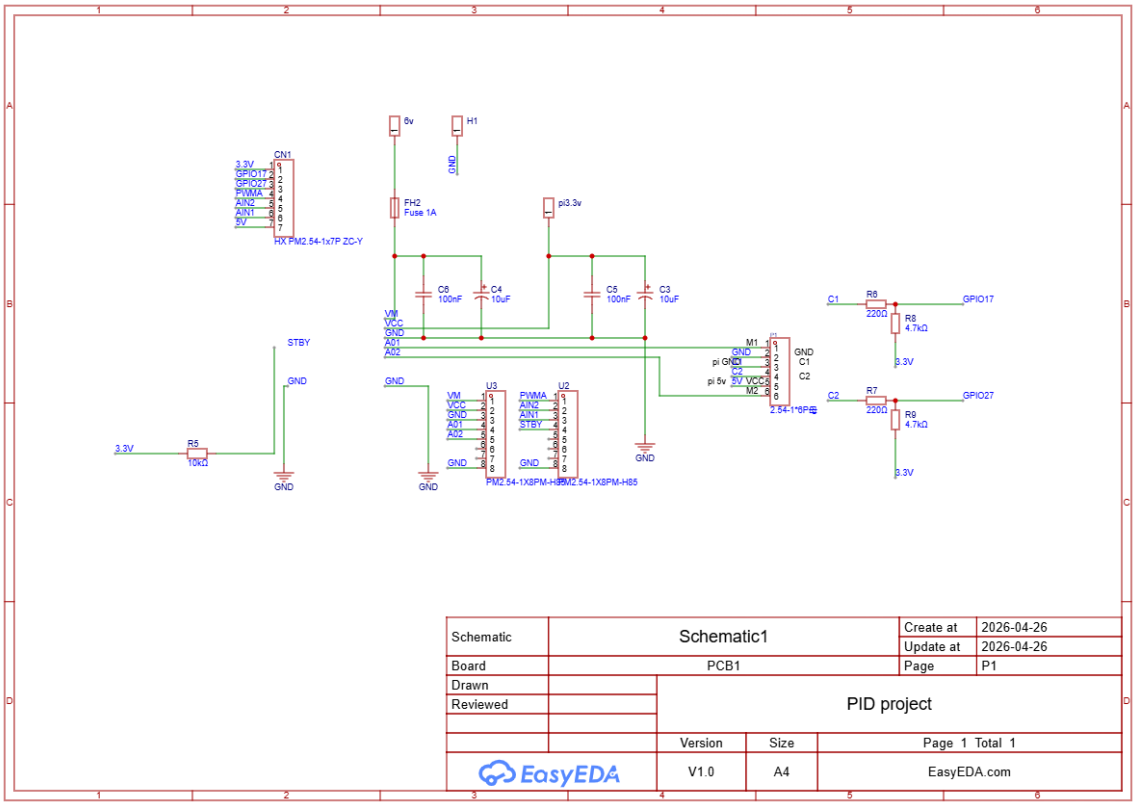
The schematic integrates the TB6612FNG motor driver, encoder signal conditioning, and power filtering components into a single system. Special attention was given to ensuring compatibility with the Raspberry Pi's 3.3 V logic levels, particularly for encoder inputs.

Key protective and supporting components included:

- Pull-up resistors for encoder signals

- Series resistors for GPIO protection
- Decoupling capacitors for noise filtering
- A fuse for overcurrent protection

This structured design ensures stable operation and simplifies PCB layout implementation.




## 3.2 Component Selection

- Pull-up resistors (4.7 kΩ): Used for encoder outputs to prevent floating signals and ensure reliable logic levels
- Series resistors (220 Ω): Added to protect Raspberry Pi GPIO pins and reduce noise spikes
- Decoupling capacitors (100 nF, 10 µF): Used to filter power supply noise and stabilize voltage levels
- Bulk capacitors: Placed across supply lines to handle sudden current demands from the motor
- Connectors and headers: Used for secure interfacing with the Raspberry Pi, motor, and power supply

## 3.3 PCB Layout Design

The PCB layout was designed to ensure reliability, low noise, and ease of assembly.

- High-current paths (motor supply and outputs) were kept short and wide to reduce resistance and heat
- The TB6612FNG driver was placed close to the motor terminals, minimizing noise and voltage drops
- Signal traces (encoder and GPIO lines) were routed separately from power lines to reduce interference

Component placement was optimized to maintain a clear and organized layout

The final PCB layout (Figure X) shows efficient routing and compact design suitable for embedded applications.

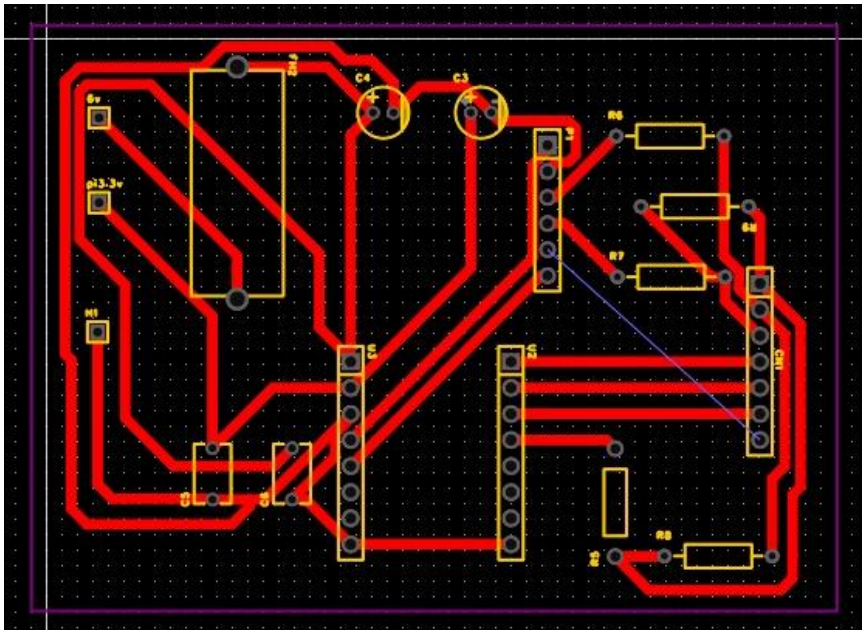
## 3.4 Design Rules and Considerations

### 3.4.1 Trace Width Calculation

Trace widths were selected based on expected current levels. Wider traces were used for motor power lines to safely carry higher current, while thinner traces were used for signal lines.

This ensures:

- Reduced voltage drop
- Lower heat generation
- Improved reliability



### 3.4.2 Grounding Strategy

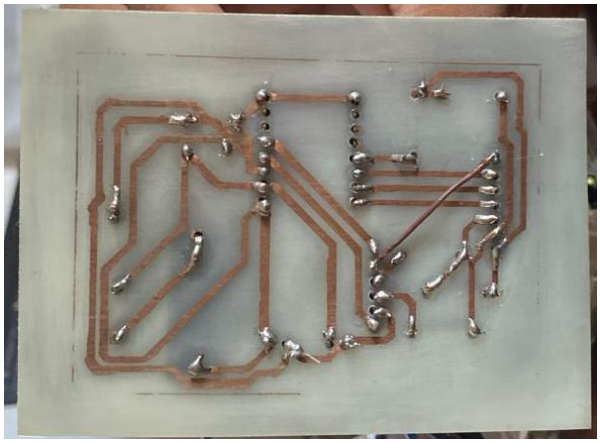
A common ground was implemented across the entire PCB to provide a stable reference for both logic and power circuits.

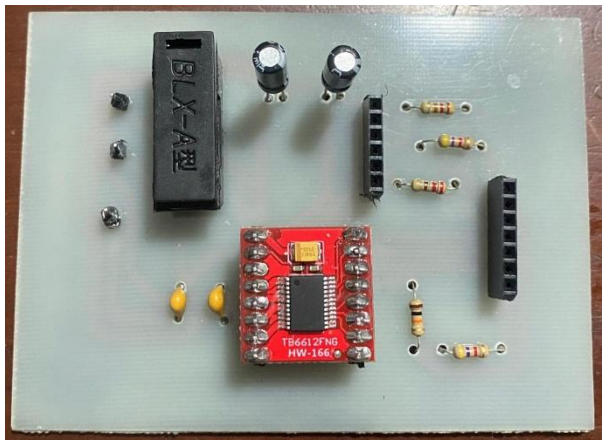
## 3.5 PCB Fabrication Process

Once the PCB design was completed, standard Gerber files were generated using the EDA tool and submitted for fabrication.

The fabrication process included:

- Verification of design rules (DRC)
- Exporting manufacturing files
- Producing the PCB using a selected fabrication service





## 3.6 Final PCB Assembly

After fabrication, all components were manually assembled onto the PCB.

- Components such as resistors, capacitors, headers, and the motor driver were soldered carefully
- Special attention was given to critical connections such as GPIO headers and power lines
- The assembled board was inspected for solder bridges and connection errors before testing

## 3.7 Power-On Verification Procedure

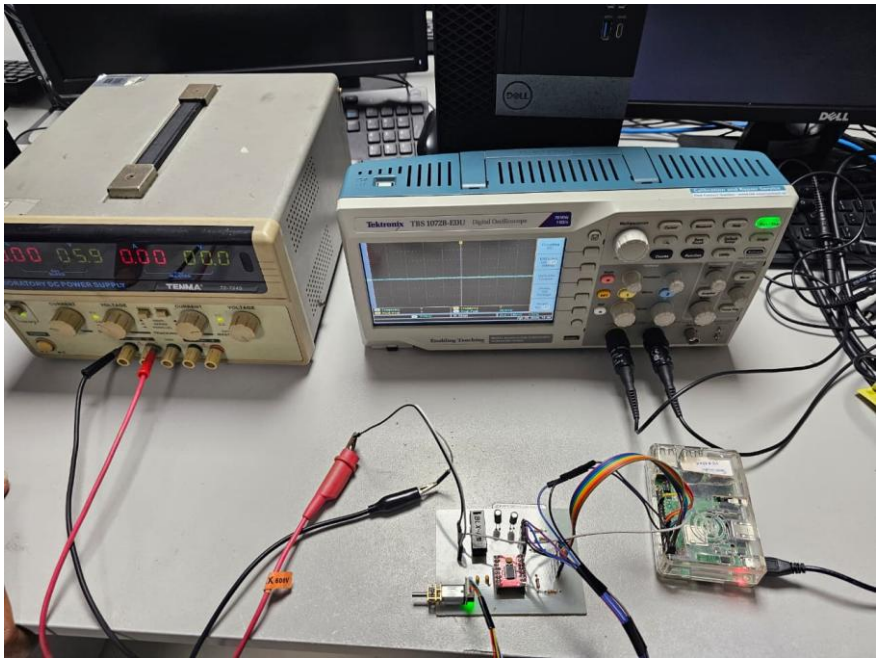
Before the finished PCB was connected to the Raspberry Pi, a strict safety verification procedure was followed to avoid damaging the microcomputer.

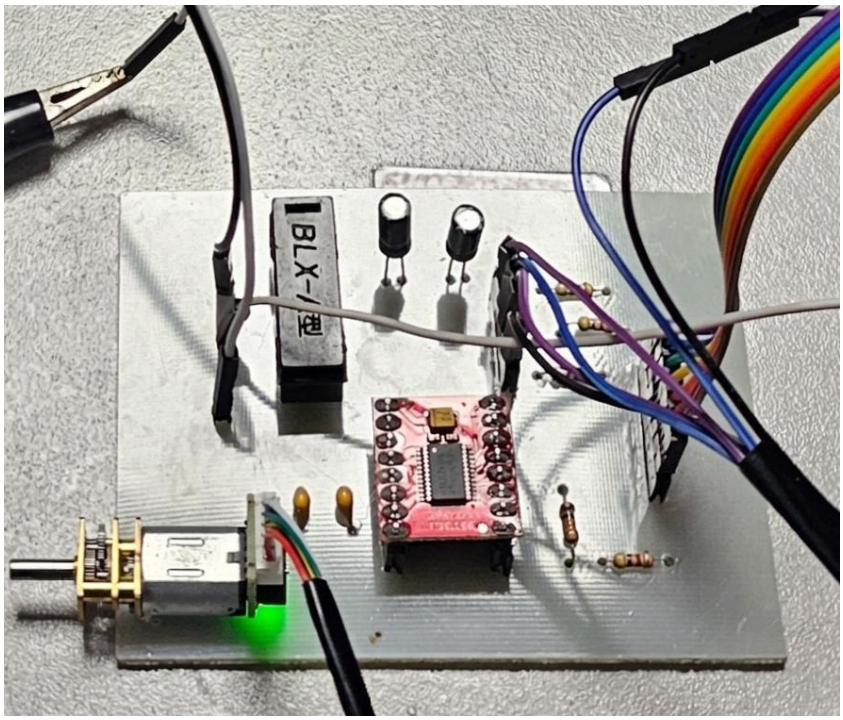
- **Dead Test (No Power):** A digital multimeter was used to perform a full continuity check. Adjacent pins were tested to confirm there were no accidental solder bridges. Most importantly, it was confirmed that the motor power rail and the 3.3 V logic rail were completely electrically isolated from each other.
- **Live Test (Logic Power Only):** The board was powered with only the 3.3 V logic supply. Encoder signal lines and the TB6612FNG standby state were verified with a multimeter. Only after the board passed this logic test was the motor supply connected and enable

## 3.8 PCB Design Deliverables

The following deliverables are included in this report and digital submission:

- Full schematic exported from EasyEDA
- PCB layout screenshot showing component placement and all copper
- Photographs of the completed, soldered, and inspected PCB
- Gerber files submitted as a ZIP archive


Tektronix TBS 1072B-EDU
DELL


BLX-I型

## 4. System Modelling & Open-Loop Characterization (Module 1)

To understand the baseline physical behaviour of the 6V DC motor before applying feedback control, its theoretical mathematical model was first examined. The dynamic behaviour of a brushed DC motor is described by a first-order transfer function relating input voltage to rotational speed:

$$G(s) = \Omega(s) / V(s) = K / (\tau s + 1)$$

**K** — DC Gain: the steady-state speed output per unit of input voltage.

**τ** — Mechanical Time Constant: the time required to reach 63.2% of the final steady-state speed.

### 4.1 PPR Calibration & Speed Calculation

Accurate speed calculation relies on precise encoder data. Physical verification confirmed the motor uses a high-resolution optical encoder producing 700 Pulses Per Revolution (PPR). Operating at a strict 50 ms (0.05 s) sampling time (dt), the raw RPM was calculated using the standard formula:

$$RPM = (Pulse\ Count \times 60.0) / (PPR \times dt)$$

This formula converts the raw encoder pulse count accumulated within each 50 ms window into a meaningful engineering unit (RPM), forming the primary feedback signal for the closed-loop PID controller.

## 4.2 Open-Loop Step Response Tests

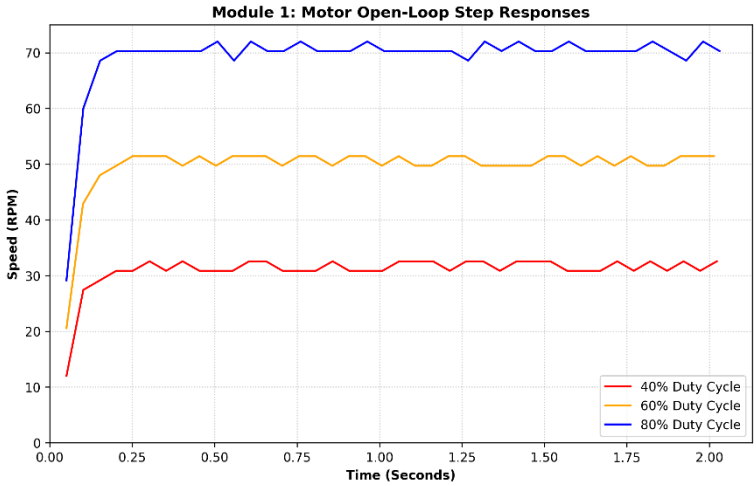
To observe how the motor behaves without feedback control, open-loop step response tests were conducted. A Python script applied a sudden, fixed PWM signal to the motor driver and logged the motor speed (RPM) versus elapsed time. Three power levels were tested: 40%, 60%, and 80% PWM duty cycle.

Table 4.1 below summarises the results of the open-loop step response tests across all three duty cycles.

***Table 4.1 — Open-Loop Step Response Summary***

| PWM Duty Cycle (%) | Approx. Steady-State Speed (RPM) | Observation |
|---|---|---|
| 40% | ~34 RPM | Lowest test speed; linear response confirmed |
| 60% | ~50.6 RPM | Reference test case; used for model derivation |
| 80% | ~67 RPM | Highest test speed; proportional scaling holds |

Analysis: The steady-state speed increases proportionally with the PWM duty cycle, which is consistent with the expected linear behaviour of a brushed DC motor within its standard operating range.




## 4.3 Time Constant & Plant Model Derivation

To extract the exact mathematical parameters for the theoretical model, the 60% PWM test data was isolated and analyzed to identify the time constant (τ). Table 4.2 summarises the derived plant parameters.

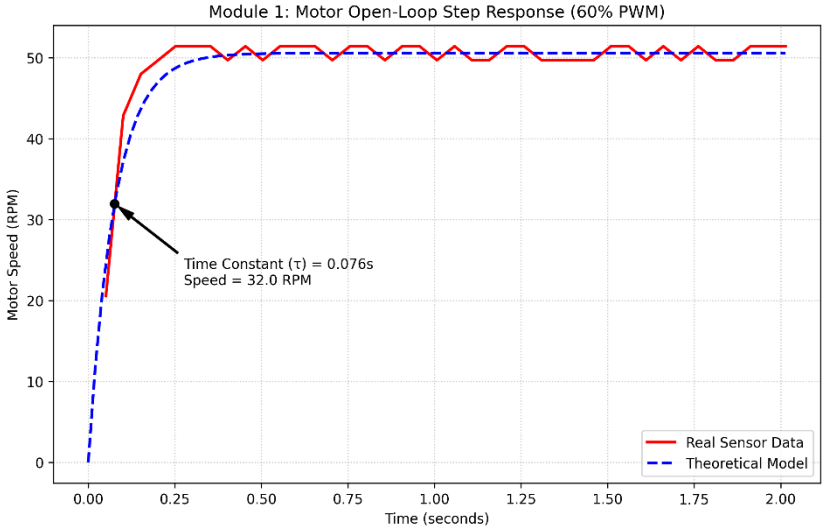


*Table 4.2 — Empirical Plant Model Parameters*

| Parameter | Symbol | Value |
|---|---|---|
| DC Gain | K | 0.843 |
| Time Constant | τ | 0.076 s |
| Steady-State Speed (60% PWM) | Ω_ss | 50.6 RPM |
| 63.2% Speed Threshold | 0.632 × Ω_ss | ≈ 32.0 RPM |
| Encoder Resolution (PPR) | PPR | 700 PPR |
| Sampling Time | dt | 0.05 s (20 Hz) |

At a 60% duty cycle, the steady-state speed was approximately 50.6 RPM. The time taken to reach 63.2% of this speed (approximately 32.0 RPM) was measured at $\tau = 0.076$ s. Based on the steady-state voltage-to-speed ratio, the DC gain was calculated as $K = 0.843$. This provided the final empirical plant model:

$$G(s) = 0.843 / (0.076s + 1)$$

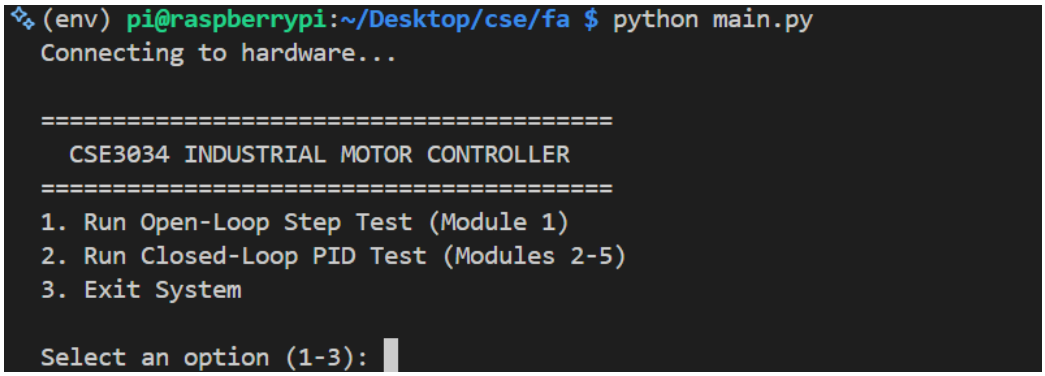
**Hardware Limitation Note:** Because the natural time constant is extremely fast (under 80 ms) and the sampling rate is 50 ms (20 Hz), the motor reaches near-maximum speed within the first two data samples. This confirmed that an aggressive, high-frequency control loop would be required to tame the acceleration.

## 5. SCADA Architecture & Software Implementation (Module 2)

Unlike standard terminal-based controllers, this system was engineered as an Industrial IoT (IIoT) application, combining a unified Command Line Interface (CLI) for hardware testing with a real-time web SCADA (Supervisory Control and Data Acquisition) dashboard for live PID tuning and monitoring.

### 5.1 Unified Application Interface

To streamline testing and data collection without constantly rewriting code, a unified main.py application was developed. Launching this script initiates a clean, interactive terminal menu, allowing the user to select operational modes seamlessly. The master script automatically handles data logging, saving CSV files, and generating the performance graphs required for analysis.

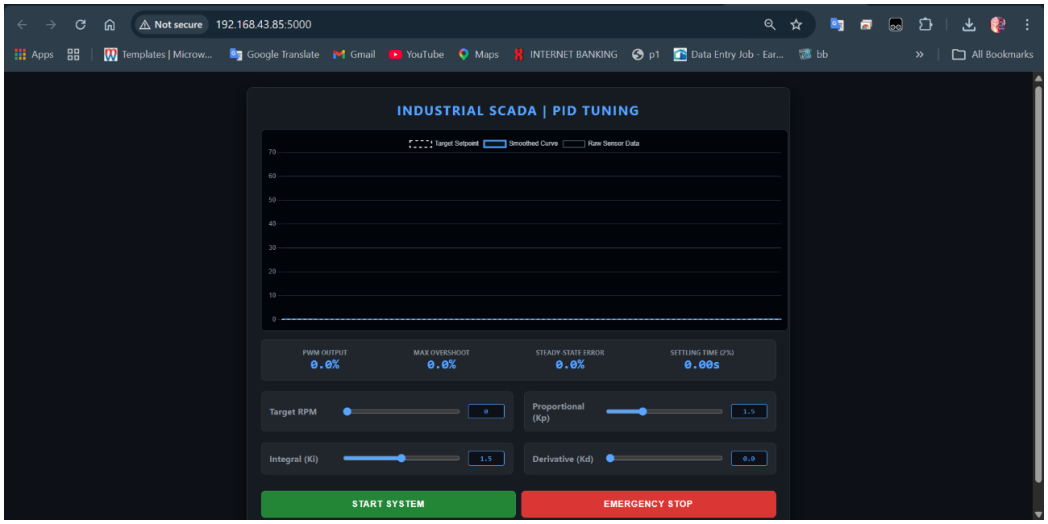


## 5.2 Live Web SCADA Dashboard

For the closed-loop implementation, a Flask-based web server was deployed on the Raspberry Pi, enabling remote monitoring and tuning via any web browser on a host PC. Table 5.1 summarises the key dashboard features.

***Table 5.1 — SCADA Dashboard Feature Summary***

| Feature | Description |
|---|---|
| Flask Web Server | Deployed on Raspberry Pi; accessible from any browser on the local network. |
| Live PID Sliders | Bidirectional WebSocket communication transmits Kp, Ki, Kd, and Target RPM in real time. |
| Live HUD Metrics | Displays Maximum Overshoot, Steady-State Error, and Settling Time computed from live telemetry. |
| CLI Test Menu | Unified main.py provides an interactive terminal menu to select operational modes and auto-save CSV logs and graphs. |
| pigpio Daemon | Hardware-level PWM generation at 1 kHz frequency for precise motor drive. |

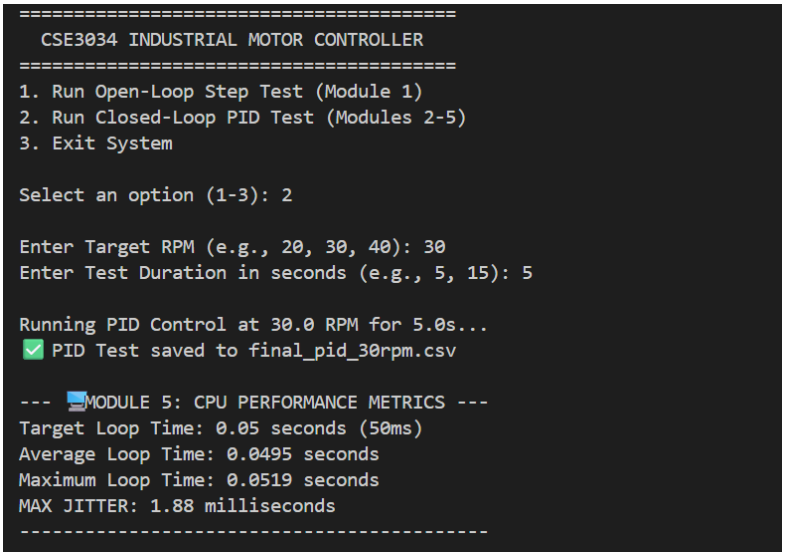


## 5.3 Control Loop Timing & OS Jitter

The control backend utilises the pigpio daemon for hardware-level timing and PWM generation at 1 kHz frequency. The main PID control loop was designed to execute at precisely 20 Hz (dt = 0.05 s).

However, because the Raspberry Pi utilises a standard Linux operating system rather than a Real-Time Operating System (RTOS), background system processes cause slight timing fluctuations. As shown in the system diagnostics, the maximum loop time occasionally stretches to 0.0519 s, resulting in a maximum CPU jitter of 1.88 milliseconds.

**Jitter Compensation:** Because PID calculus relies heavily on dt for integration and derivation, the software dynamically calculates the exact elapsed time using time.time() during every loop cycle, ensuring mathematical accuracy despite OS scheduling delays.



## 5.4 Digital Signal Processing — Moving Average Filter

The 700-PPR digital encoder counts in discrete whole numbers, resulting in quantization noise — small, jagged mathematical spikes in the RPM calculation even when motor speed is constant. If fed directly into the PID equation, the Derivative term amplifies these spikes, causing violent PWM oscillations ("derivative kick").

To resolve this, a 10-point centered moving average filter was implemented in software. This DSP technique smooths the feedback signal before it enters the PID algorithm, sacrificing a negligible amount of phase lag for massive gains in mechanical stability.

## 5.5 Anti-Windup & Safety Mechanisms

To prevent the Integral term from accumulating infinite error during motor startup or physical stalling (integrator windup), the accumulator was strictly clamped to a maximum value of ±200.0. Furthermore, the entire execution block was wrapped in a try...finally software constraint, guaranteeing that in the event of a script timeout, unexpected crash, or emergency stop command, the PWM output is immediately grounded to 0%, safely halting the motor.

## 6. Systematic Gain Tuning Study (Module 3)

A structured heuristic tuning method was utilised to determine the optimal gains for a 30 RPM setpoint. The process progressed through four distinct testing phases, isolating each mathematical term to observe its physical effect on the plant. Table 6.1 provides a complete summary of all four tuning tests.

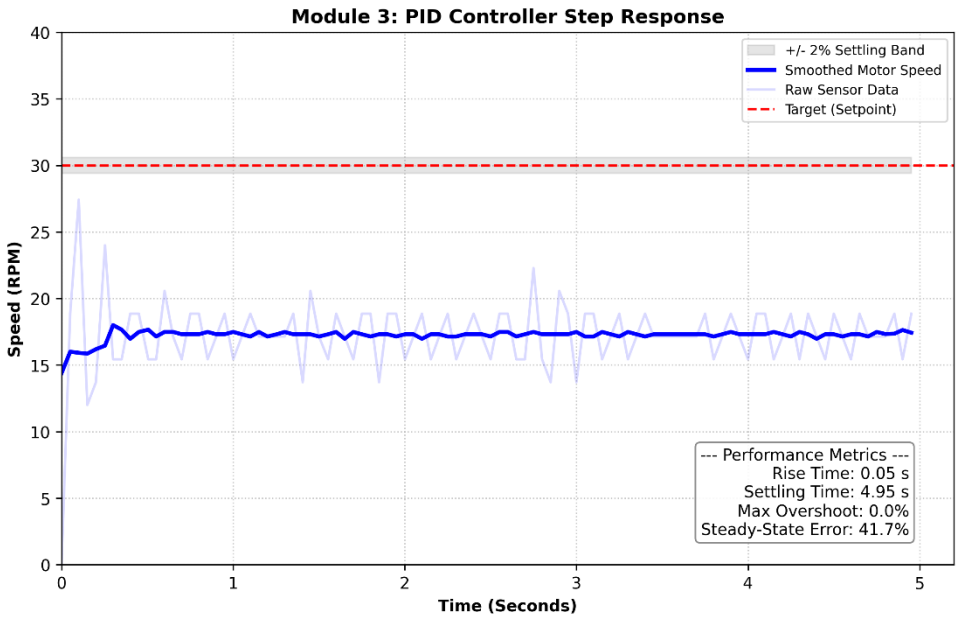
***Table 6.1 — Systematic PID Gain Tuning Progression***

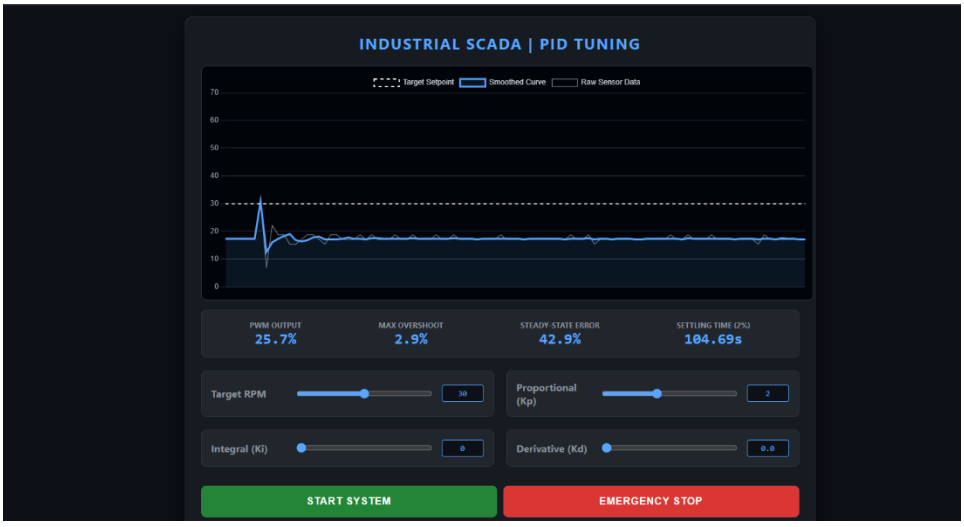
| Test Phase | Kp | Ki | Kd | Behaviour Observed | Key Finding |
|---|---|---|---|---|---|
| Test 1 — P Only | 2.0 | 0.0 | 0.0 | High steady-state error (41.7%); instability present | Integral term required to overcome friction and lock onto setpoint |
| Test 2 — PI Control | 1.0 | 1.5 | 0.0 | Setpoint reached; minor overshoot and longer settling time | P and I terms competing; further Kp reduction required |
| Test 3 — Refined PI | 0.5 | 1.5 | 0.0 | Controlled rise; near-zero steady-state error; no oscillations | Integral term smoothly dictates setpoint approach |
| **Test 4 — Optimal PID** | **0.5** | **1.6** | **0.01** | Tight steady-state lock; minimal overshoot; stable PWM output | Optimal balance of speed, stability, and noise immunity |

## Test 1 — Initial Baseline (Proportional-Only)

**Parameters:** Kp = 2.0 | Ki = 0.0 | Kd = 0.0

A high proportional gain was applied to force a rapid initial response. While the motor accelerated quickly, it exhibited a high steady-state error of 41.7% and instability. The muscle of the P-term alone was sufficient to initiate movement but failed to lock onto the precise 30 RPM setpoint, proving that an Integral term was mathematically required to overcome steady-state mechanical friction.

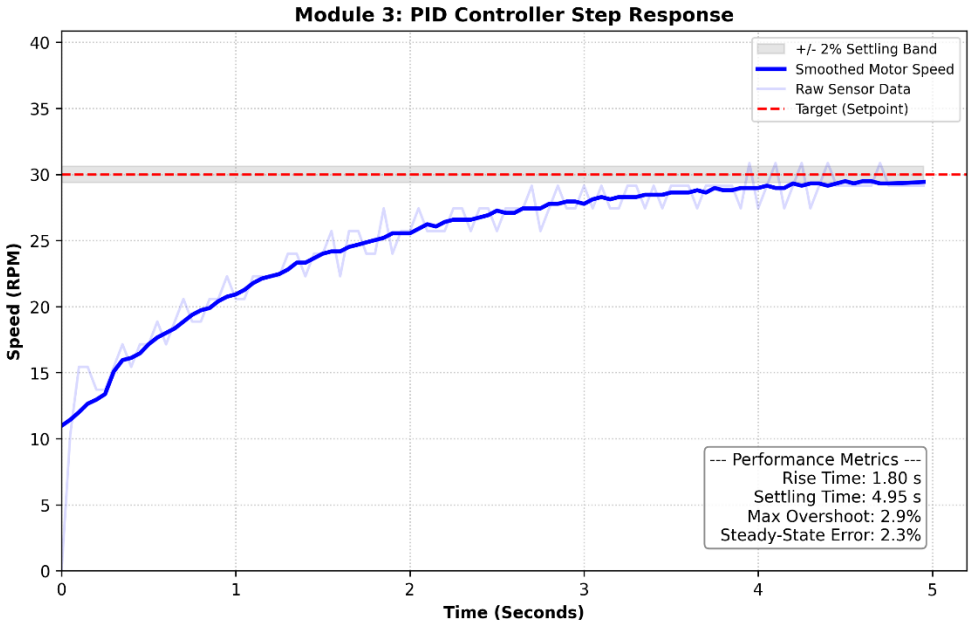


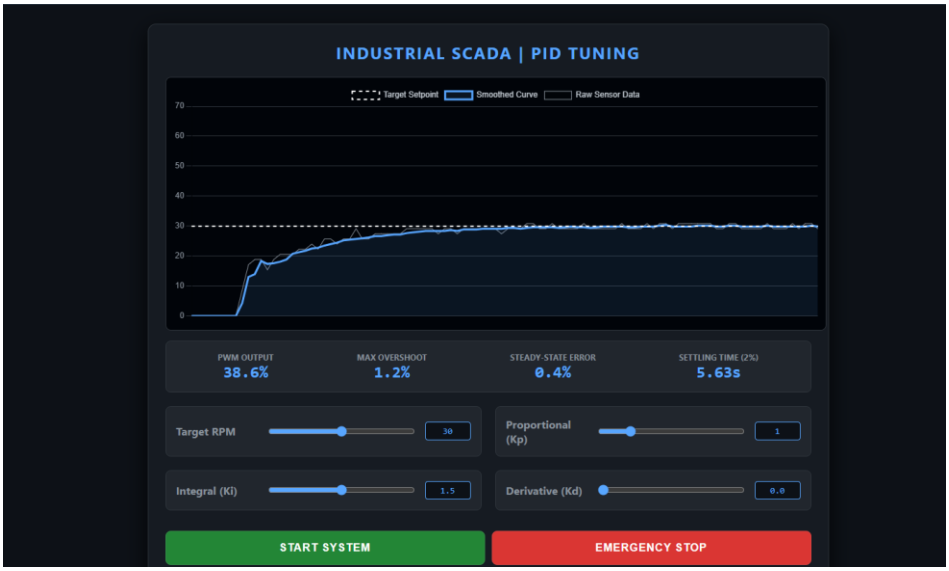



## Test 2 — Introducing Integral Action (PI Control)

**Parameters:** Kp = 1.0 | Ki = 1.5 | Kd = 0.0

The Proportional gain was halved to 1.0 to reduce aggression, and an Integral gain of 1.5 was introduced to eliminate the steady-state error. The controller successfully reached the 30 RPM setpoint. However, the step response was slightly unoptimised, with the transient curve showing minor overshoot and a longer settling time as the P and I terms competed.

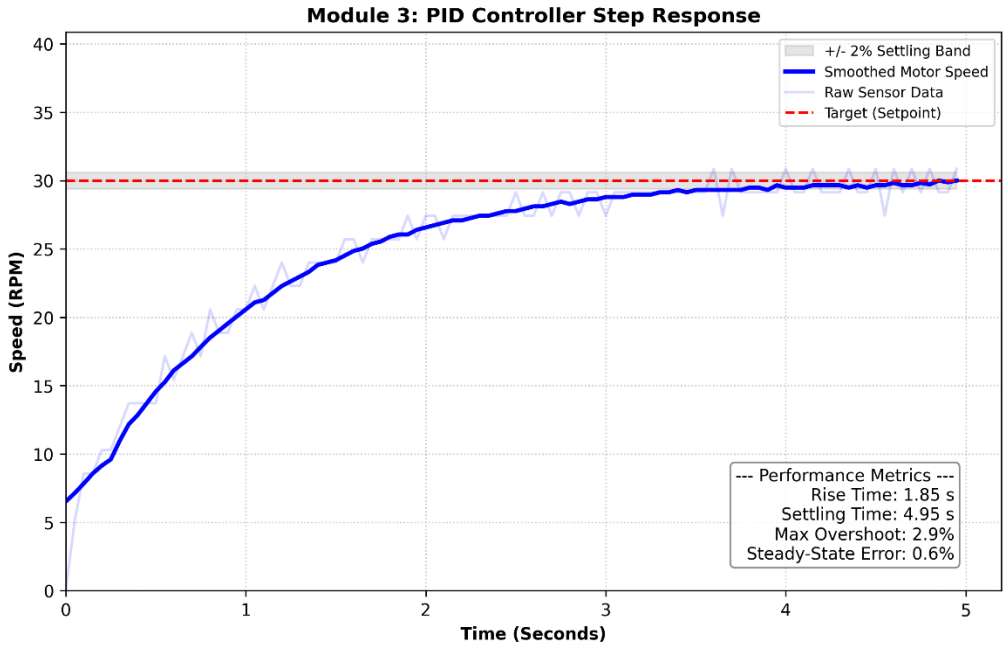


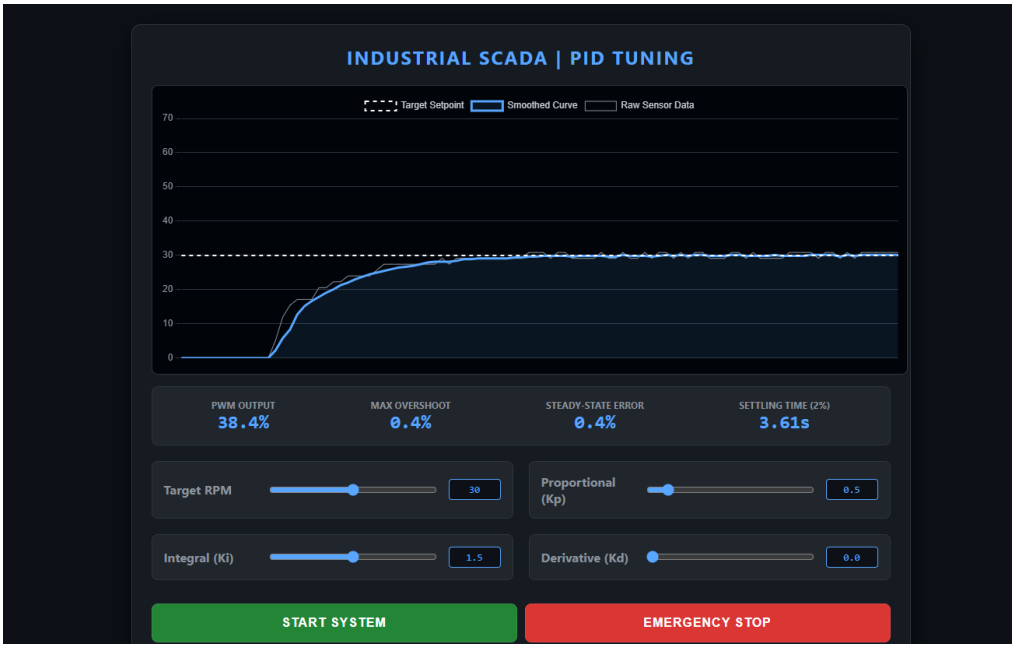



## Test 3 — Refining the PI Controller

**Parameters:** Kp = 0.5 | Ki = 1.5 | Kd = 0.0

Kp was further reduced to 0.5. This was a critical optimisation. Lowering the proportional strength allowed the Integral term to smoothly dictate the approach to the setpoint. The rise time became highly controlled, and the steady-state error was virtually eliminated without inducing violent oscillations in the gearbox.

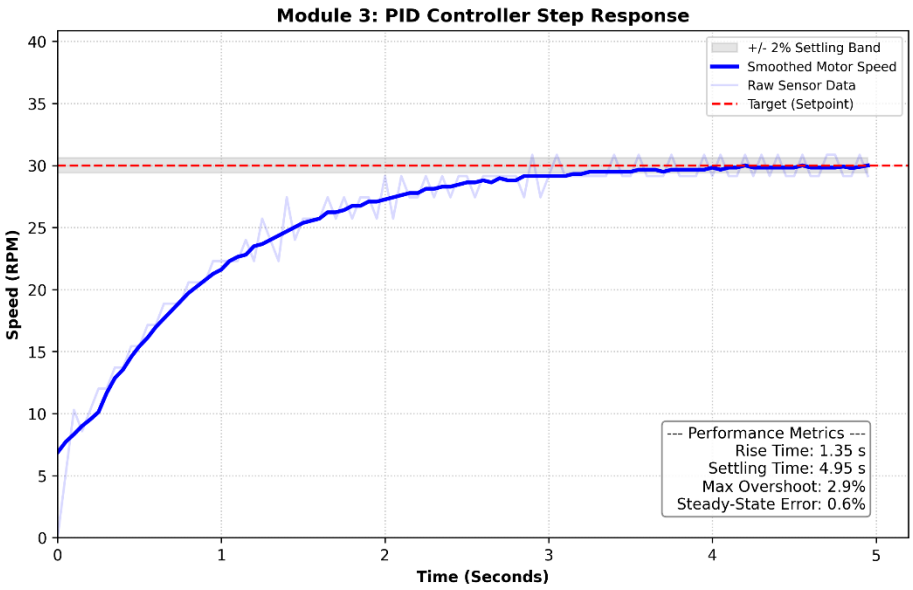


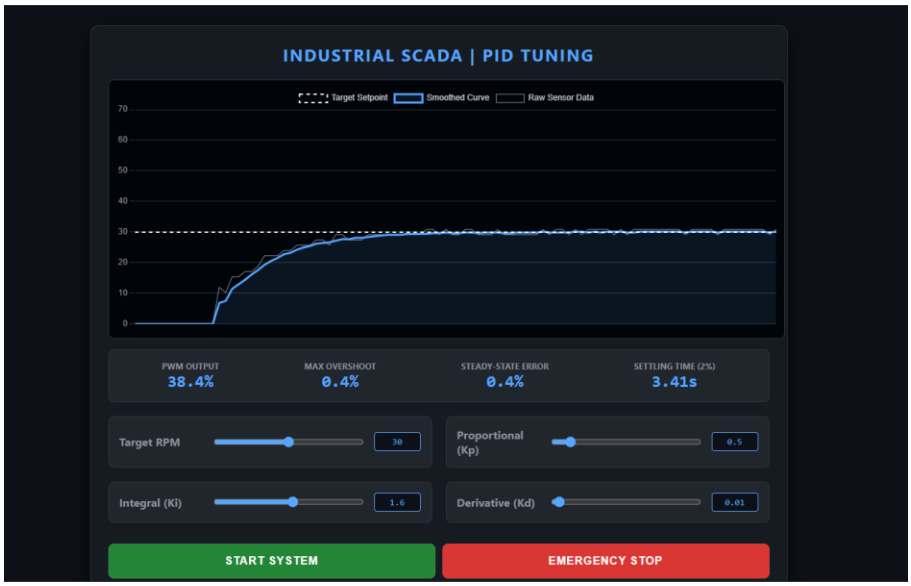



## Test 4 — Final Optimal Tuning

**Parameters:** Kp = 0.5 | Ki = 1.6 | Kd = 0.01

For the final optimisation, Ki was micro-adjusted to 1.6 to tighten the steady-state lock, and a minimal Derivative term (0.01) was added to provide slight damping against overshoot. Kd was kept near zero intentionally; higher values reacted poorly with the remaining sensor noise, causing erratic PWM spikes. This configuration represented the absolute optimal balance of speed and stability.



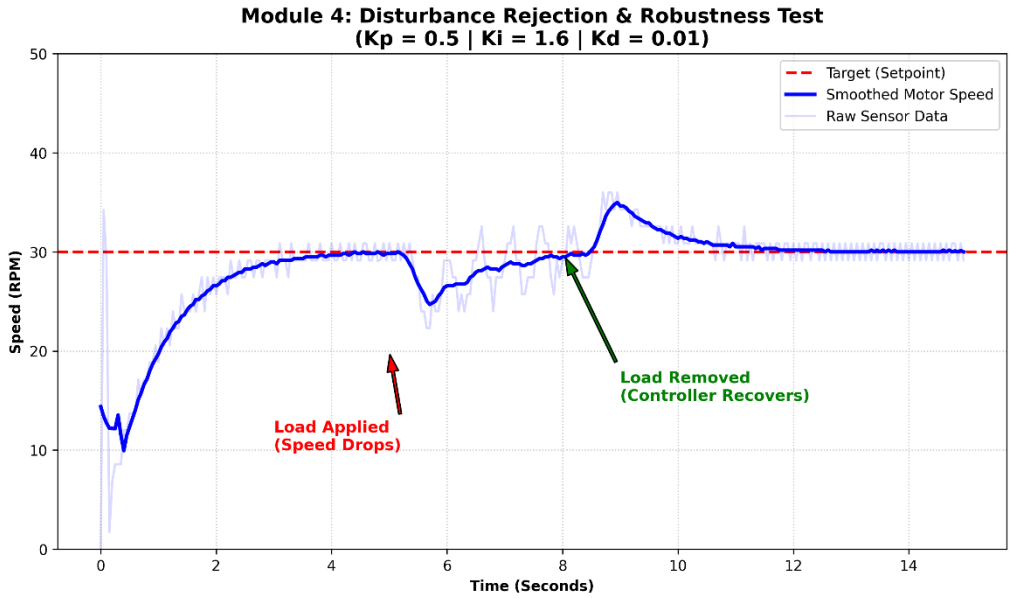
## 7. Disturbance Rejection & Robustness Testing (Module 4)

A control system is only viable for industrial deployment if it can reject external load disturbances. To validate the optimised Kp = 0.5, Ki = 1.6, Kd = 0.01 configuration, a physical load test was conducted at the 30 RPM setpoint.

The motor was allowed to settle perfectly at 30 RPM before a manual friction load was applied to the spinning shaft. The following sequence of events was recorded:

- RPM immediately dropped below the 30 RPM setpoint upon load application.
- The Integral term accurately accumulated the sustained error and spiked the PWM output to overcome the resistance.
- Upon abrupt load removal, the built-up integral energy caused a brief, predicted overshoot.
- The controller rapidly shed the accumulated error, returning the motor to within the 2% settling band.

This confirms the controller possesses excellent disturbance rejection capabilities suitable for dynamic industrial environments.

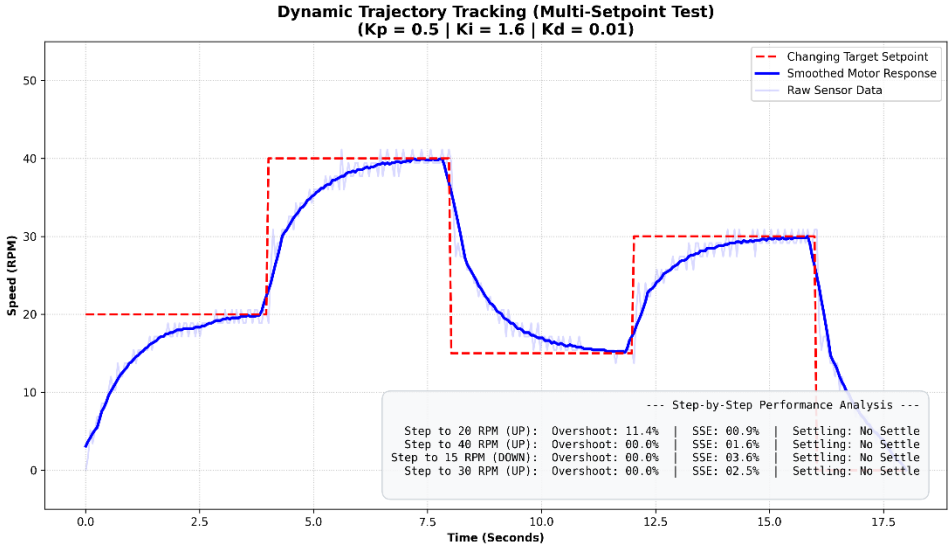


## 7.2 Multi-Setpoint Trajectory Tracking

To confirm that the optimised PID gains were not overfitted to just the 30 RPM setpoint, the controller was subjected to an automated staircase trajectory test. The system was programmed to track a dynamic sequence of speeds: 20 RPM → 40 RPM → 15 RPM → 30 RPM. Table 7.2 details each step in the trajectory.

***Table 7.2 — Multi-Setpoint Trajectory Step Analysis***

| Step | From (RPM) | To (RPM) | Direction | Key Behaviour |
|---|---|---|---|---|
| 1 | 0 | 20 | Step Up | Initial startup; controlled rise to first setpoint |
| 2 | 20 | 40 | Aggressive Step Up | Rapid acceleration; minimal overshoot; P-term scales effectively |
| 3 | 40 | 15 | Large Step Down | Anti-windup clamp (±200) prevents undershoot & stall |
| 4 | 15 | 30 | Step Up | Returns to nominal setpoint; |




This test validates the true stability of the controller across the motor's entire operational range. The following key behaviours were observed:

- **Stepping Up (20 → 40 RPM):** When commanding aggressive step-ups, the controller delivered rapid acceleration with minimal overshoot, proving the Proportional term scales effectively across different speed ranges.
- **Stepping Down (40 → 15 RPM):** This transition serves as the ultimate test for the software's Anti-Windup clamp (±200). If the Integral memory had not been properly constrained, stepping down would cause a violent undershoot, potentially stalling the motor or reversing its direction. Instead, the controller elegantly unwound its integral accumulation and safely settled at the lower speed — validating the anti-windup implementation.
- **Return to Nominal (15 → 30 RPM):** The controller returned to the original 30 RPM setpoint with near-zero steady-state error, confirming that gain performance is consistent and not dependent on the direction or magnitude of the prior step.

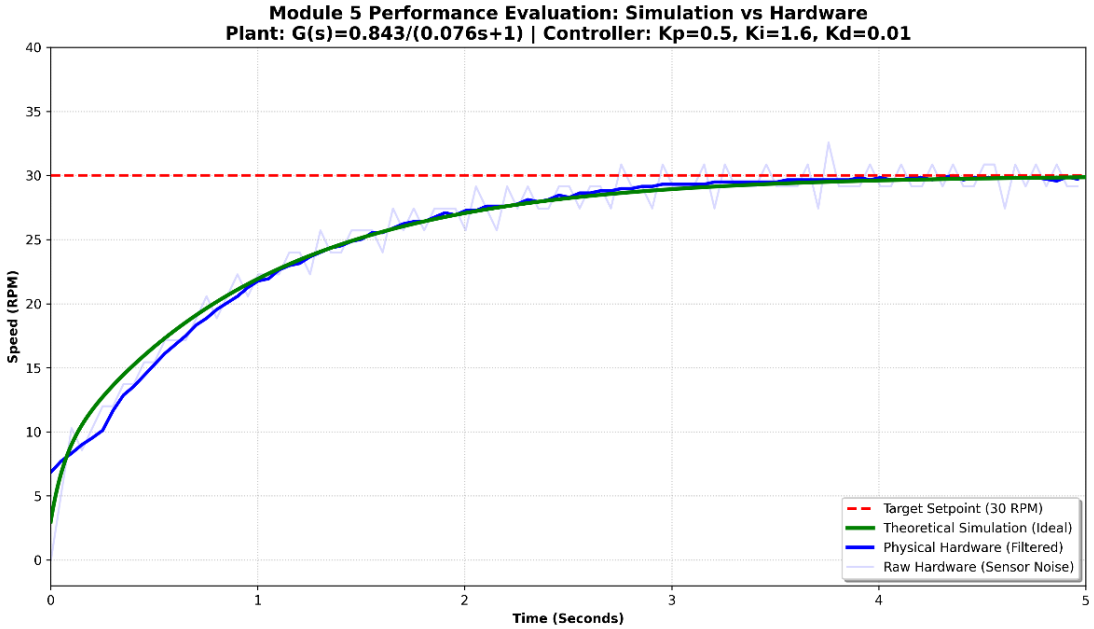
## 8. Performance Evaluation & Non-Idealities (Module 5)

While the mathematical tuning produced an optimal physical response, comparing the empirical hardware data against the theoretical mathematical model reveals the realities of deploying control systems on non-ideal hardware.

### 8.1 Simulation vs. Hardware Comparison

When plotting the theoretical transfer function response against the physical hardware response, slight deviations in the transient curve are expected. The theoretical simulation assumes continuous, perfect calculation capabilities and instantaneous voltage application. The physical hardware response is constrained by the 100% PWM saturation limit during startup and the physical stiction of the gears.

Despite these non-linearities, the hardware closely tracked the theoretical model and achieved the identical steady-state target of 30 RPM, successfully validating the Module 1 plant model derived in Section 4.



## 8.2 Summary of Results and Physical Limitations

Table 8.1 provides a structured summary of all physical non-idealities encountered throughout the project, their root causes, and the engineering mitigations applied.

***Table 8.1 — Non-Idealities, Causes, and Mitigations***

| Non-Ideality | Cause | Mitigation Applied |
|---|---|---|
| Encoder Quantization Noise | 700-PPR encoder outputs discrete integer counts causing RPM spikes | 10-point centered moving average filter applied before PID input |
| OS Scheduling Jitter | Standard Linux OS causes loop time fluctuation (max +1.88 ms) | Dynamic dt calculation using time.time() on every loop cycle |
| Integrator Windup | Integral term accumulates unbounded error during stall or startup | Integral accumulator clamped to ±200.0; try...finally safety block |
| PWM Saturation (Hardware) | Physical 100% PWM ceiling during startup limits theoretical response | Anti-windup clamp prevents overdriving; model validated against hardware |
| Gear Stiction | Static friction in gearbox creates nonlinear dead-zone at low speeds | Integral term accumulates sufficient energy to overcome stiction threshold |

By acknowledging and compensating for these physical limitations, the final controller successfully maintained a strict 30 RPM setpoint with negligible steady-state error, validating its readiness for deployment in a real-world automated system.

## 9. Conclusion

In this project, a complete real-time PID control system and Industrial SCADA dashboard were successfully designed, built, and optimised for a 6V DC motor using a Raspberry Pi. Through methodical heuristic tuning and advanced software design — including real-time jitter compensation and digital signal filtering — a final optimised PID controller was achieved:

***Final Parameters:  Kp = 0.5  |  Ki = 1.6  |  Kd = 0.01***

The system surpassed the standard assignment constraints by integrating a live web-based SCADA UI, providing a professional-grade interface for monitoring overshoot, steady-state error, and settling time in real time. Robustness testing proved the system can recover from severe physical load disturbances seamlessly.

The primary lesson learned during this project is the critical difference between perfect mathematical theory and real-world hardware implementation. Specific findings include:

- A fast mechanical time constant ($\tau = 0.076$ s) relative to the sampling period ($dt = 0.05$ s) presents fundamental challenges for digital control that must be addressed in software.
- Digital signal processing (moving average filter) is essential when the Derivative term is used with quantized encoder feedback.
- Anti-windup protection is a non-negotiable safety mechanism in any practical integrating controller.
- OS-level jitter on Linux must be compensated through dynamic dt measurement rather than assuming a fixed time step.

Ultimately, the system is highly stable, robust, and completely prepared for practical application in an industrial setting.

## Codes Link with Full project on GitHub

All codes and generated plots and csv files under open repository on GitHub

GitHub link: https://github.com/pkurukuladithya/cse.git